## THM_LABS

# CRACK THE HASH

*HASH CRACKING FUNDAMENTALS EVERY JUNIOR PENTESTER SHOULD UNDERSTAND*

**Prepared by**

**Ahmed Abdelaziz Eldkrory (EDGAR)**

**Junior Penetration Tester**

**ahmed.a.eldkrory@gmail.com**

**www.linkedin.com/in/ahmed-eldkrory**

# Overview

This document demonstrates my understanding of **hashing and password-cracking fundamentals** from a security perspective and summarizes key concepts and techniques covered in the **Crack the Hash**.

The focus of this write-up is on **technical reasoning, learning outcomes, and ethical cybersecurity practices**, rather than reproducing lab instructions or solutions directly from the platform.

Full write-up:
https://medium.com/@ahmed.a.eldkrory/crack-the-hash-tryhackme-room-15efde35f418

Room Link:
https://tryhackme.com/room/crackthehash


# Crack the Hash

The goal of this room is to introduce the **fundamentals of password hashing and hash cracking**, which are essential topics for anyone starting in penetration testing or cybersecurity.

Hashing is a cryptographic process that converts input data (such as a password) into a **fixed-length string of characters**. The output, known as a **hash**, is designed to represent the original data without revealing it. Ideally, this process is **one-way**, meaning the original input cannot be retrieved directly from the hash.

For example, when a user creates an account on a website, the system typically **stores the hash of the password rather than the password itself**. During login, the entered password is hashed again and compared to the stored hash.

This approach significantly improves security because even if a database is compromised, attackers do not immediately obtain the original passwords.

However, some older hashing algorithms — such as **MD5** and **SHA1** — are now considered insecure due to weaknesses and the availability of powerful cracking tools.

Understanding how hashes work and how they can be cracked is a fundamental skill for security professionals.

# Why Hashing Matters in Cybersecurity

Hashing plays a critical role in many areas of cybersecurity, including:

- Password storage
- Data integrity verification
- Digital signatures
- Authentication systems

For penetration testers, understanding hashing allows them to:

- Analyze compromised credential databases
- Perform password auditing during security assessments
- Identify weak hashing algorithms used by systems
- Demonstrate real-world risks associated with poor password practices

Learning how attackers approach password cracking also helps defenders design **stronger authentication systems**.

---

# Core Concept: Hashing

Hashing refers to the process of transforming data into a **fixed-size output using a mathematical function**.

Key characteristics of cryptographic hashes include:

- **Deterministic** – the same input always produces the same hash
- **Fixed length** – regardless of input size
- **One-way function** – difficult to reverse
- **Collision resistant** – difficult to produce two inputs with the same hash

A useful way to visualize hashing is to imagine a **secure one-way door**:

You can pass information through it to produce a hash, but you cannot simply reverse the process to retrieve the original data.

Because of this property, hashing is widely used for **password storage and verification**.

---

# Room Structure

The room is divided into **two main levels**, each focusing on a different approach to working with hashes.

# Level 1 – Using Online Decryption Tools

The first section focuses on identifying hashes and using online databases or lookup services to recover weak passwords.

Many common hashes have already been cracked and stored in **public hash databases**, making them searchable online.

Some useful tools include:

- https://www.dcode.fr/en

These tools compare a given hash against known cracked values.

This method works well when:

- The password is weak
- The hash exists in public databases
- The algorithm is outdated

However, it does not work for **strong passwords or properly salted hashes**.

---

# Level 2 – Using Hashcat

The second section introduces **Hashcat**, one of the most powerful password-cracking tools used in cybersecurity.

Hashcat works by attempting to **guess possible passwords** and hashing them to compare with the target hash.

It supports multiple attack techniques, including:

- Dictionary attacks
- Brute-force attacks
- Mask attacks
- Rule-based attacks

Security professionals often use Hashcat during:

- Password auditing
- Red team engagements
- Security research
- Incident investigations

Understanding how these attacks work helps defenders implement **strong password policies and secure hashing algorithms**.

---

# Identifying Hashing Algorithms

Before attempting to crack a hash, it is important to determine **which algorithm was used**.

Different algorithms produce different hash formats and lengths.

Common identification methods include:

- Online hash analyzers
- Tools such as **hashid** or **hash-identifier**
- Pattern recognition based on hash length and format

Example tools:

- https://www.tunnelsup.com/hash-analyzer/

Correctly identifying the algorithm is essential because password-cracking tools require the **correct mode parameter** for each hash type.

---

# Pentester Mindset

When encountering hashes during a penetration test, security professionals typically follow a structured process:

1. **Identify the hashing algorithm**
2. **Check public databases for known hashes**
3. **Use password-cracking tools when necessary**
4. **Analyze password strength and security impact**

The objective is not just to crack passwords, but to **evaluate the security posture of the system**.

A weak hashing algorithm or easily cracked password can indicate:

- Poor password policies
- Lack of salting
- Weak authentication design

These findings help organizations strengthen their security.

# Common Challenges & Key Insights

Many beginners encounter several challenges when learning about hash cracking:

- Confusion when identifying hashing algorithms
- Misunderstanding the difference between hashing and encryption
- Overreliance on tools without understanding the underlying concepts

A key takeaway from this room is that **tools are only effective when combined with solid foundational knowledge**.

Strong understanding of hashing concepts enables security professionals to make better decisions during assessments.

---

# Key Takeaways

- Hashing is a one-way cryptographic process used to protect sensitive data
- Weak hashing algorithms such as MD5 can often be cracked
- Online databases can reveal hashes for commonly used passwords
- Tools like Hashcat enable advanced password-cracking techniques
- Identifying the hashing algorithm is a critical first step
- Understanding password security is essential for both attackers and defenders

---

# Real-World Relevance

In professional cybersecurity work, these skills support:

- Password auditing during penetration tests
- Analysis of compromised credential databases
- Security assessments of authentication systems
- Incident response investigations involving credential leaks

Ultimately, mastering these fundamentals provides the foundation for more advanced topics such as:

- Credential attacks
- Password spraying
- Hash cracking at scale
- Authentication bypass techniques

---

Every advanced offensive security technique builds on these fundamental concepts.

# Crack the Hash – Level 1: Basic Hash Cracking

## Context

In this task from the **Crack the Hash**, the focus is on understanding how weak passwords can be recovered from hashes using publicly available tools and databases.

Hashing is widely used to protect stored passwords. Instead of storing plaintext passwords, systems store **hash values** generated through cryptographic functions. During authentication, the system hashes the entered password and compares it to the stored hash.

However, if the hashing algorithm is weak or the password itself is commonly used, attackers can often recover the original value using **precomputed hash databases** or lookup tools.

This task demonstrates how easily weak credentials can be exposed when strong password practices and modern hashing techniques are not implemented.

---

## Security Perspective

For penetration testers and security professionals, the ability to analyze and crack password hashes is an important assessment skill.

During real-world engagements, testers may obtain password hashes from:

- Database leaks
- Misconfigured backup files
- Compromised systems
- Credential dumps

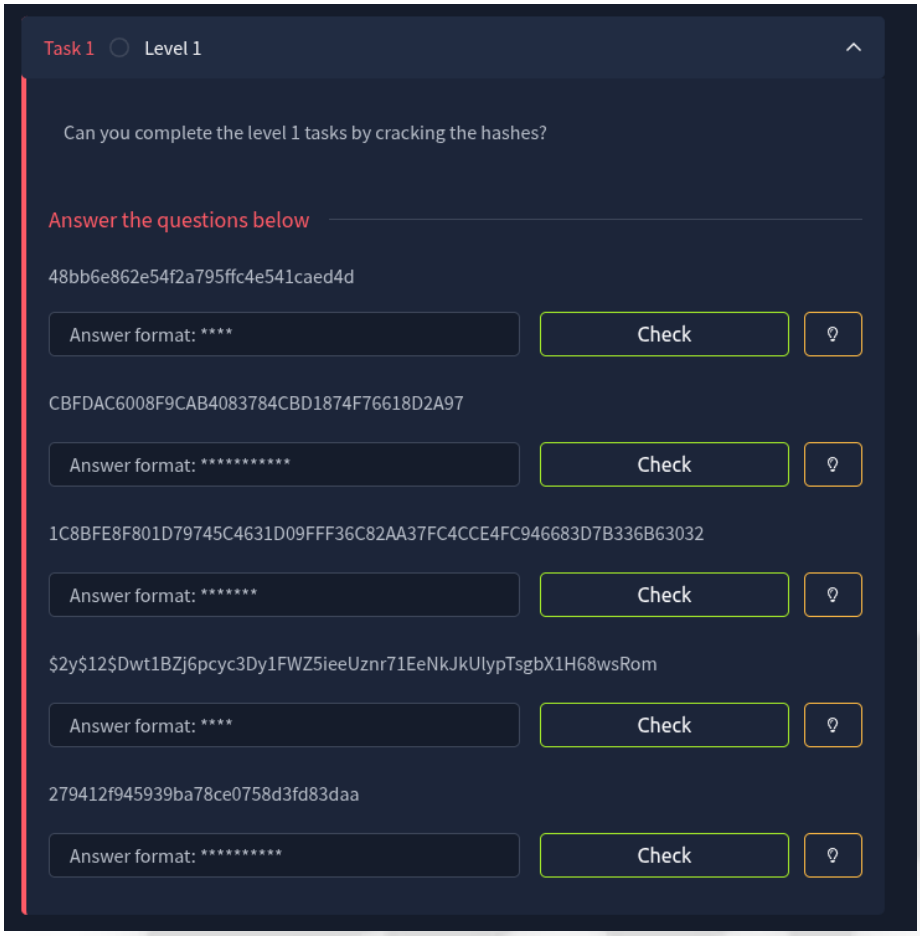
Understanding how to identify the hashing algorithm and attempt password recovery allows security professionals to:

- Evaluate password strength
- Identify weak authentication practices
- Demonstrate the impact of poor password policies
- Recommend stronger hashing algorithms such as bcrypt or Argon2

This knowledge also helps defenders understand **how attackers exploit weak credential storage mechanisms**.

---

# Task 1




The objective of this task is to recover the plaintext values of several provided hashes.

This is done by identifying the hashing algorithm used and then using online lookup tools or hash databases to find matching plaintext passwords.

The hashes in this level represent **common password patterns**, making them suitable for quick recovery through publicly available cracking services.

---

# Q&A

## Question 1

**Hash**

`48bb6e862e54f2a795ffc4e541caed4d`

## Method

First, I used an online hash analyzer from **dcode** to identify the hashing algorithm.


The tool detected that the hash uses the **MD5 algorithm**.

Next, I used the decode feature on the same website to search for the plaintext value.


## Answer

`easy`

## Explanation

The hash corresponds to the plaintext password **easy**, which already exists in public hash databases. Weak passwords like this are easily recoverable when fast hashing algorithms such as **MD5** are used.

# Question 2

## Hash

`CBFDAC6008F9CAB4083784CBD1874F76618D2A97`

## Method

First, I used the **hashid** command to identify the hashing algorithm.

```
┌──(edgar㉿kali)-[~]
└─$ hashid CBFDAC6008F9CAB4083784CBD1874F76618D2A97
Analyzing 'CBFDAC6008F9CAB4083784CBD1874F76618D2A97'
[+] SHA-1
[+] Double SHA-1
[+] RIPEMD-160
[+] Haval-160
[+] Tiger-160
[+] HAS-160
[+] LinkedIn
[+] Skein-256(160)
[+] Skein-512(160)
```

The command indicated that the hash is likely **SHA-1**.

Next, I searched the hash using **CrackStation** to check if it exists in public password databases.


## Answer

`password123`

### Explanation

The hash corresponds to the plaintext password **password123**. Since this password is extremely common, its **SHA-1 hash already exists in public rainbow tables**, allowing it to be recovered instantly.

---

# Question 3

### Hash

```
1C8BFE8F801D79745C4631D09FFF36C82AA37FC4CCE4FC946683D7B336B63032
```

### Method

I followed the same approach used in the previous question.

First, I used hashid to identify the hashing algorithm.

```
┌──(edgar㉿kali)-[~]
└─$ hashid 1C8BFE8F801D79745C4631D09FFF36C82AA37FC4CCE4FC946683D7B336B63032
Analyzing '1C8BFE8F801D79745C4631D09FFF36C82AA37FC4CCE4FC946683D7B336B63032'
[+] Snefru-256
[+] SHA-256
[+] RIPEMD-256
[+] Haval-256
[+] GOST R 34.11-94
[+] GOST CryptoPro S-Box
[+] SHA3-256
[+] Skein-256
[+] Skein-512(256)
```

The result suggested that the hash is SHA-256.

Then, I searched the hash using CrackStation to check if the plaintext password exists in public databases.

### Answer

`letmein`

### Explanation

The hash corresponds to the common password **letmein**. Even though **SHA-256** is a stronger algorithm than MD5 or SHA-1, weak and widely used passwords can still be recovered if they already exist in public hash databases.

---

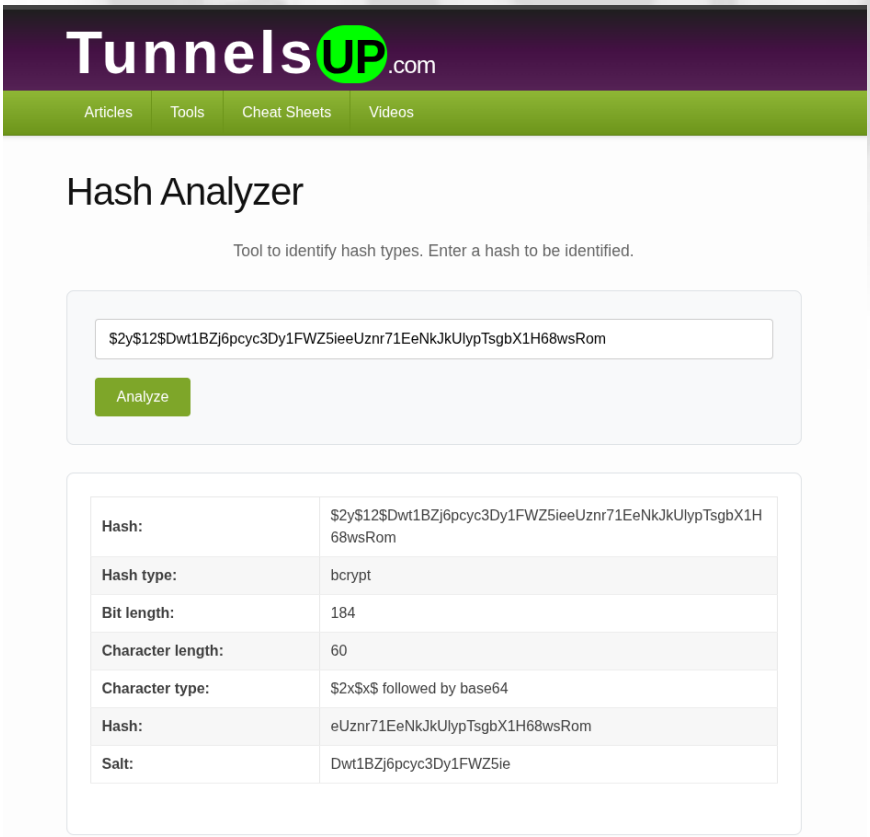
## Question 4

### Hash

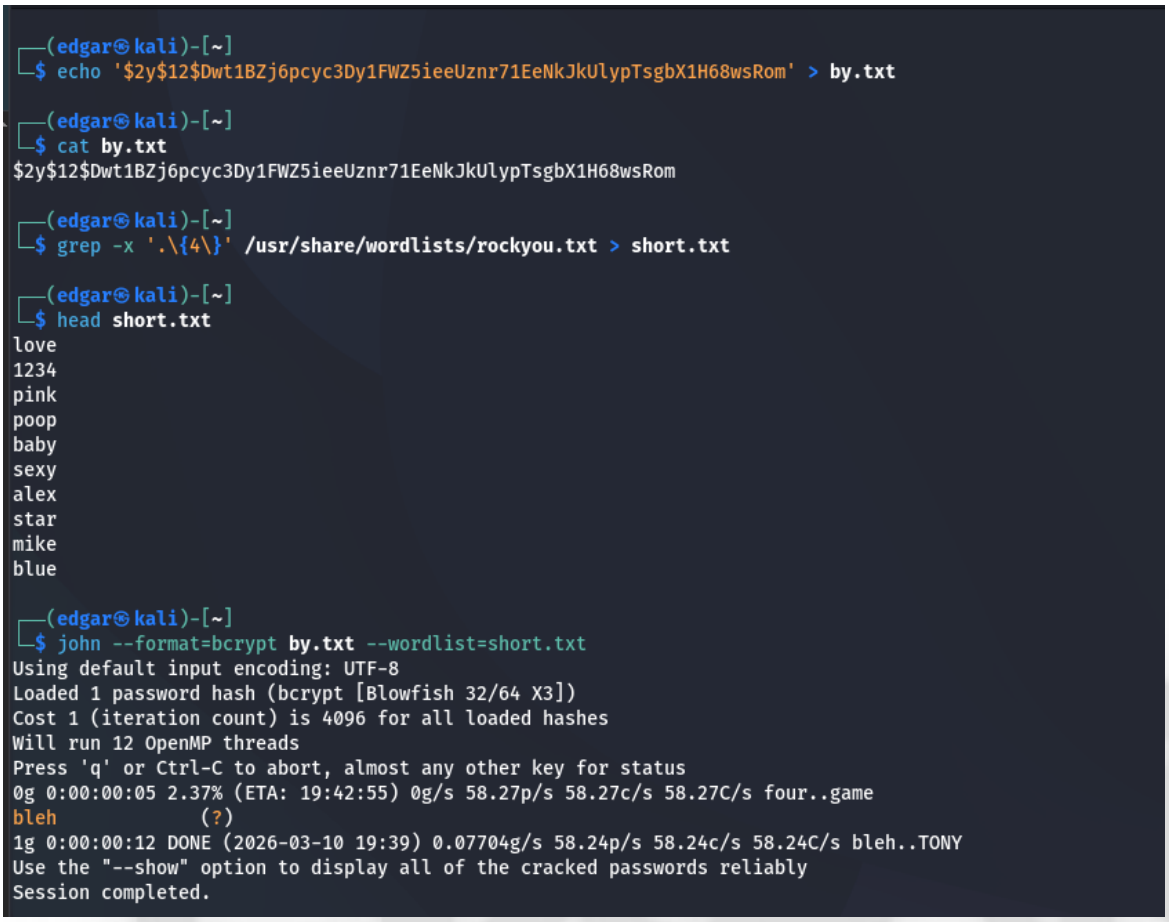
`$2y$12$Dwt1BZj6pcyc3Dy1FWZ5ieeUznr71EeNkJkUlypTsgbX1H68wsRom`

### Method

First, I used **TunnelsUp Hash Analyzer** to identify the hashing algorithm, which was detected as **bcrypt**.



After that, I prepared the cracking environment using several Linux commands.




**Command Purpose**

- **echo** → used to save the hash into a file (`by.txt`) so it can be used by cracking tools.
- **cat** → used to read the **rockyou wordlist**.
- **grep** → used to filter only **4-letter passwords** from the wordlist.
- **head** → used to preview the generated wordlist (`short.txt`).
- **john** → used to perform the password cracking using the **bcrypt format** and the custom wordlist.

## Answer

`bleh`

## Explanation

The bcrypt hash was cracked using **John the Ripper** with a filtered wordlist containing four-letter passwords. Since the correct password **bleh** exists in the list, the tool successfully recovered it.

---

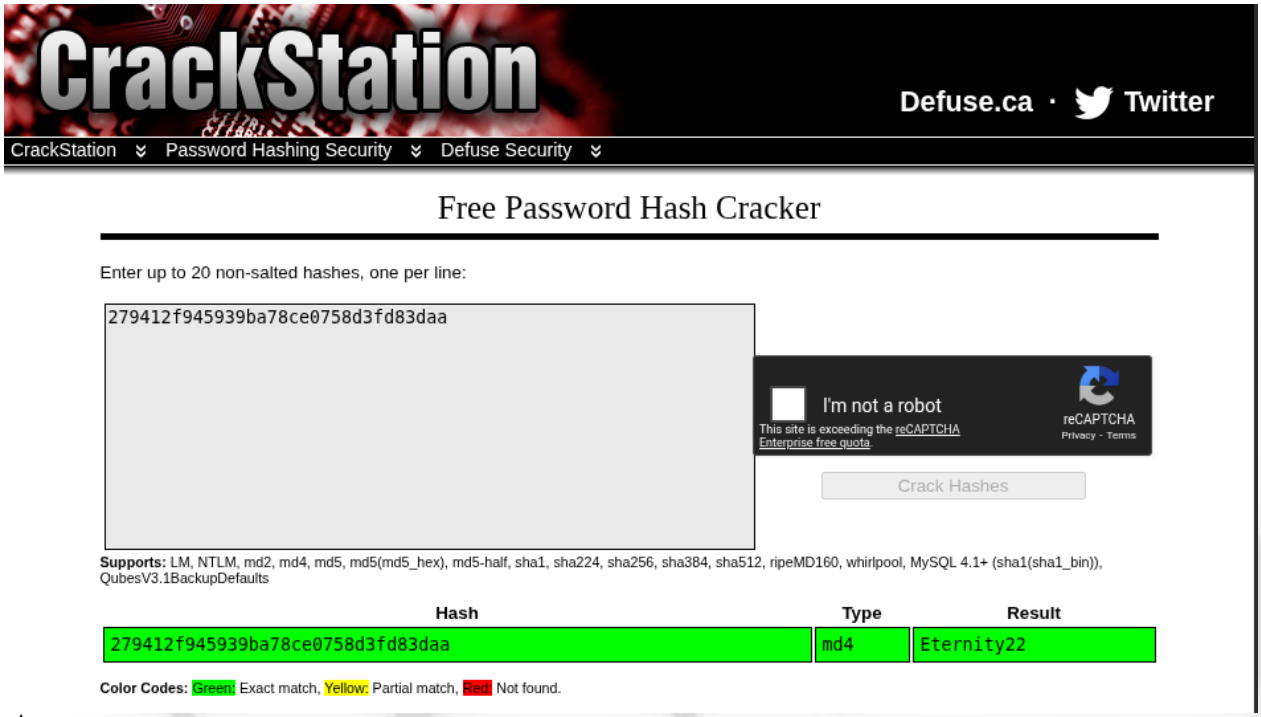
# Question 5

## Hash

`279412f945939ba78ce0758d3fd83daa`

## Method

First, I searched the hash using **CrackStation**, which compares hashes against large databases of previously cracked passwords.




## Answer

`Eternity22`

## Explanation

The hash matched an existing entry in CrackStation's database, revealing the plaintext password **Eternity22**. This demonstrates how hashes generated from common or previously leaked passwords can often be recovered instantly using public hash lookup services.

---

## Key Learning Insight

This task reinforces an important lesson in cybersecurity:

**Password security depends on both strong hashing algorithms and strong user passwords.**

Even secure cryptographic functions cannot fully protect systems if weak or commonly used passwords are allowed. Security professionals must therefore focus on:

- Strong password policies
- Secure hashing algorithms
- Salting mechanisms
- Credential monitoring and auditing

---

## Crack the Hash – Level 2

## Context

Level 2 of the Crack the Hash increases the difficulty compared to Level 1. Instead of relying mainly on online hash lookup services, this stage requires the use of dedicated password-cracking tools.

In real security assessments, analysts often encounter hashes that are not present in public databases, which means they must attempt password recovery using dictionary attacks with large wordlists.

One of the most commonly used wordlists in cybersecurity training is rockyou.txt, which contains millions of real passwords collected from historical breaches.

---

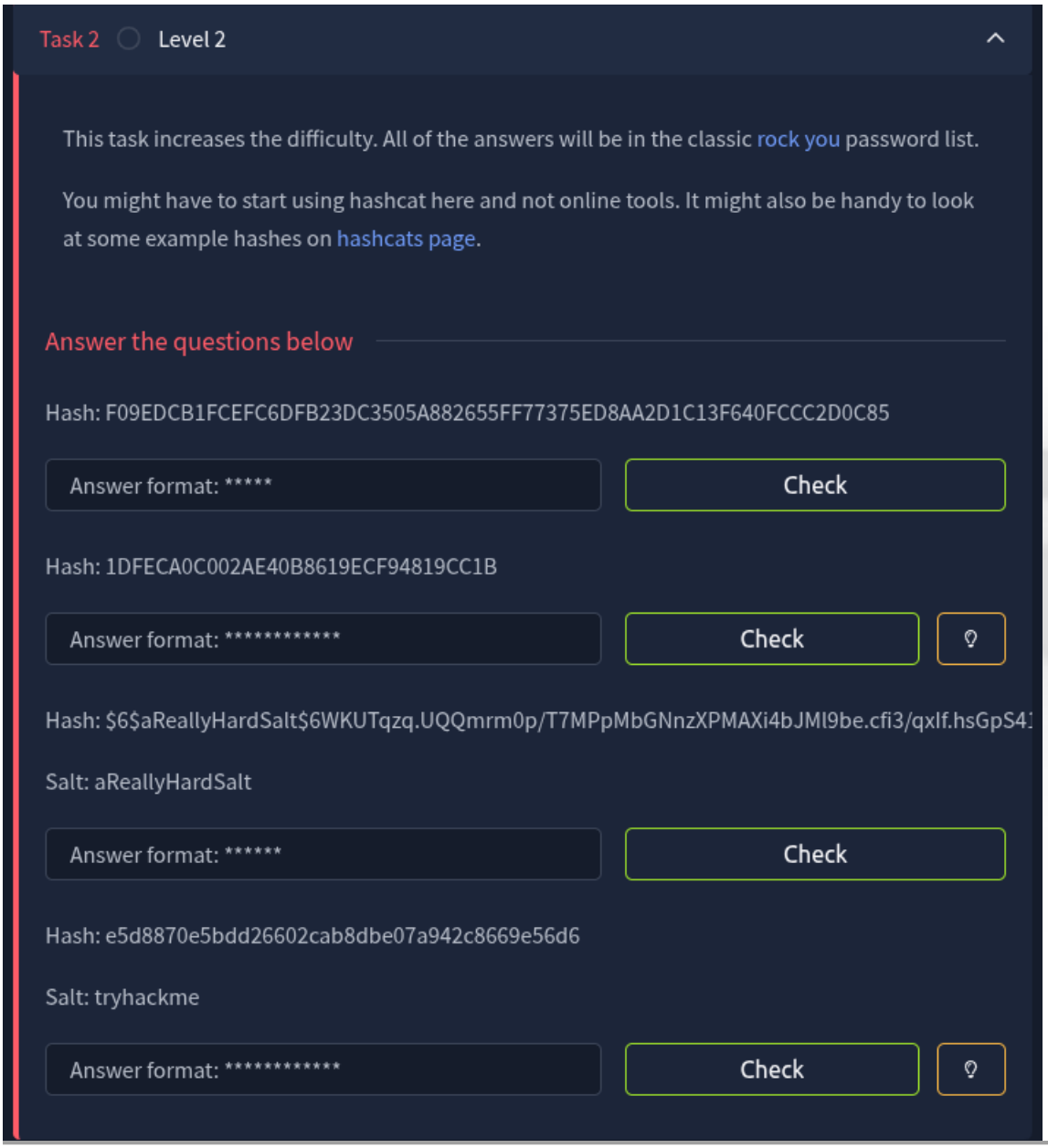
## Security Perspective

For penetration testers, using password-cracking tools is essential when performing:

- Credential auditing
- Password strength assessments
- Post-exploitation analysis
- Incident response investigations
- Tools such as Hashcat allow security professionals to test password strength efficiently using different attack techniques.
- Understanding how to configure these tools and choose the correct hash mode is an important practical skill.

---

# Task 2


Task 2
Level 2
This task increases the difficulty. All of the answers will be in the classic rock you password list.
You might have to start using hashcat here and not online tools. It might also be handy to look at some example hashes on hashcats page.
Answer the questions below
Hash: F09EDCB1FCEFC6DFB23DC3505A882655FF77375ED8AA2D1C13F640FCCC2D0C85
Answer format: *****
Check
Hash: 1DFECA0C002AE40B8619ECF94819CC1B
Answer format: **********
Check
Hash: $6$aReallyHardSalt$6WKUTqzq.UQQmrm0p/T7MPpMbGNnzXPMAXi4bJMl9be.cfi3/qxlf.hsGpS41
Salt: aReallyHardSalt
Answer format: ******
Check
Hash: e5d8870e5bdd26602cab8dbe07a942c8669e56d6
Salt: tryhackme
Answer format: **********
Check

# Q&A

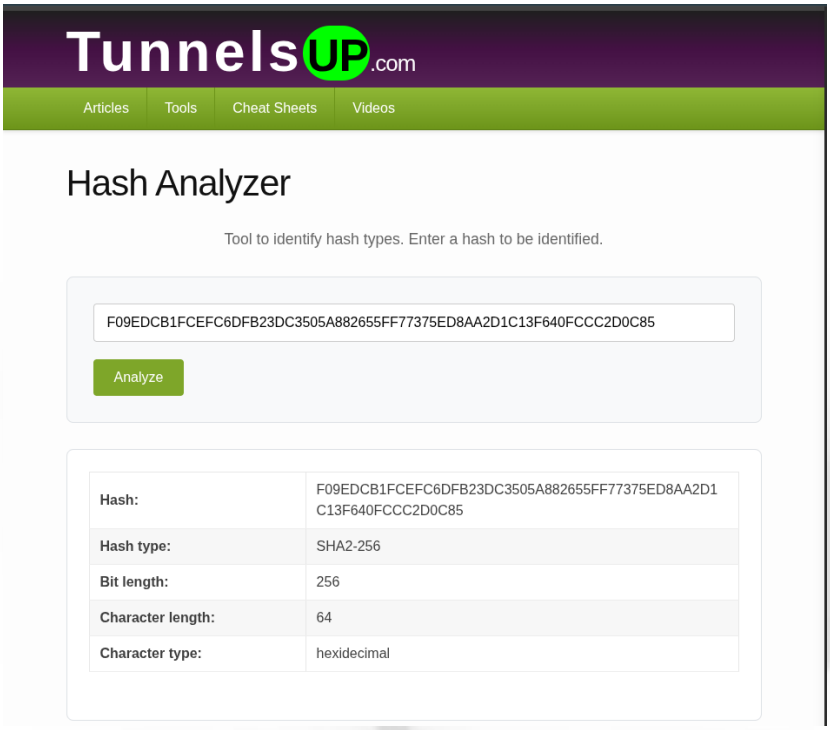
# Question 1

## Hash

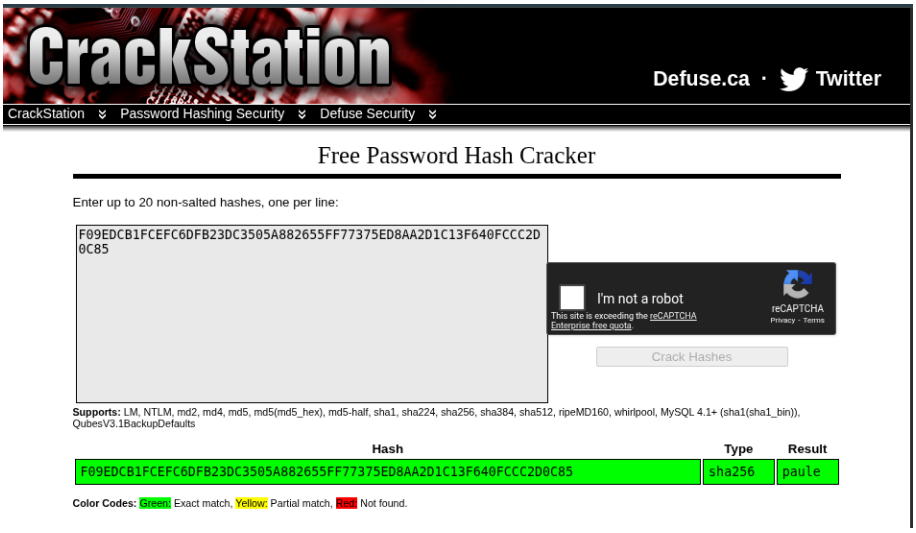
```
F09EDCB1FCEFC6DFB23DC3505A882655FF77375ED8AA2D1C13F640FCCC2D0C85
```

## Method

First, I used **TunnelsUp Hash Analyzer** to identify the hashing algorithm.




After identifying the hash type, I searched the hash using **CrackStation** to check if it exists in public password databases.



### Answer

`paule`

### Explanation

The hash matched an entry in CrackStation's database, revealing the plaintext password **paule**. This demonstrates how hashes generated from passwords present in common wordlists such as **rockyou** can often be recovered quickly using public hash lookup services.

---

## Question 2

### Hash

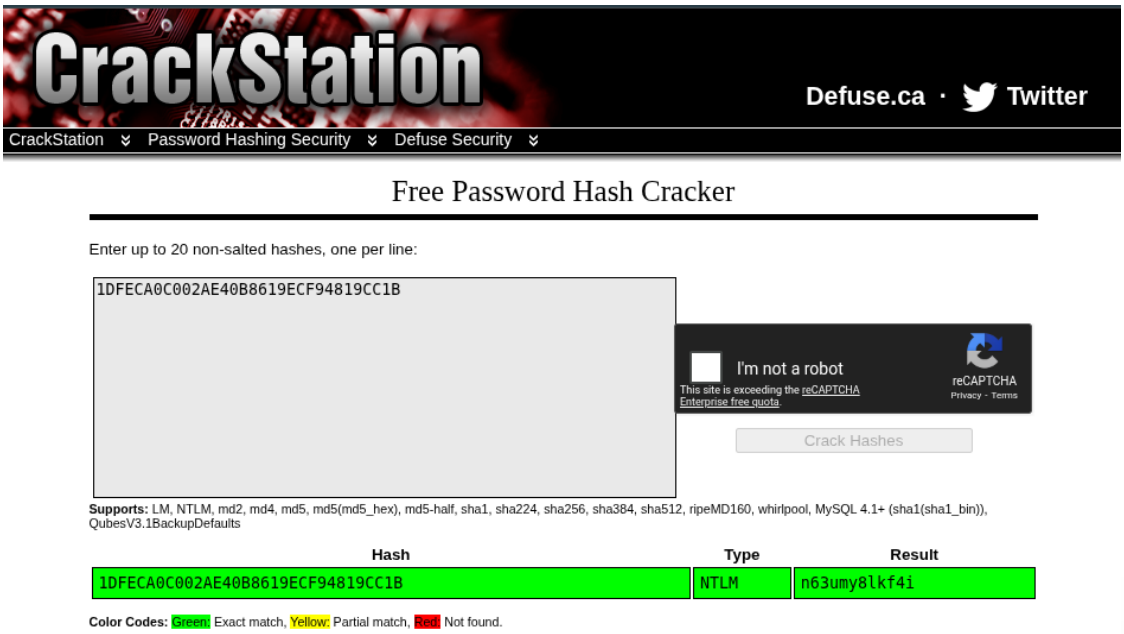
`1DFECA0C002AE40B8619ECF94819CC1B`

### Method

First, I used **hashid** to determine the hashing algorithm.

```
┌──(edgar㉿kali)-[~]
└─$ hashid 1DFECA0C002AE40B8619ECF94819CC1B
Analyzing '1DFECA0C002AE40B8619ECF94819CC1B'
[+] MD2
[+] MD5
[+] MD4
[+] Double MD5
[+] LM
[+] RIPEMD-128
[+] Haval-128
[+] Tiger-128
[+] Skein-256(128)
[+] Skein-512(128)
[+] Lotus Notes/Domino 5
[+] Skype
[+] Snefru-128
[+] NTLM
[+] Domain Cached Credentials
[+] Domain Cached Credentials 2
[+] DNSSEC(NSEC3)
[+] RAdmin v2.x
```

After identifying the hash type, I searched it using **CrackStation** to check if the plaintext password exists in public hash databases.




### Answer

`n63umy8lkf4i`

### Explanation

CrackStation matched the hash with the plaintext password **n63umy8lkf4i**. This shows that even less common passwords can sometimes be recovered if they already exist in large password databases derived from leaked credentials.

---

# Question 3

### Hash

`$6$aReallyHardSalt$6WKUTqzq.UQQmrm0p/T7MPpMbGNnzXPMAXi4bJMl9be.cfi3/qxIf.hsGpS41BqMhSrHVXgMpdjS6xeKZAs02.`

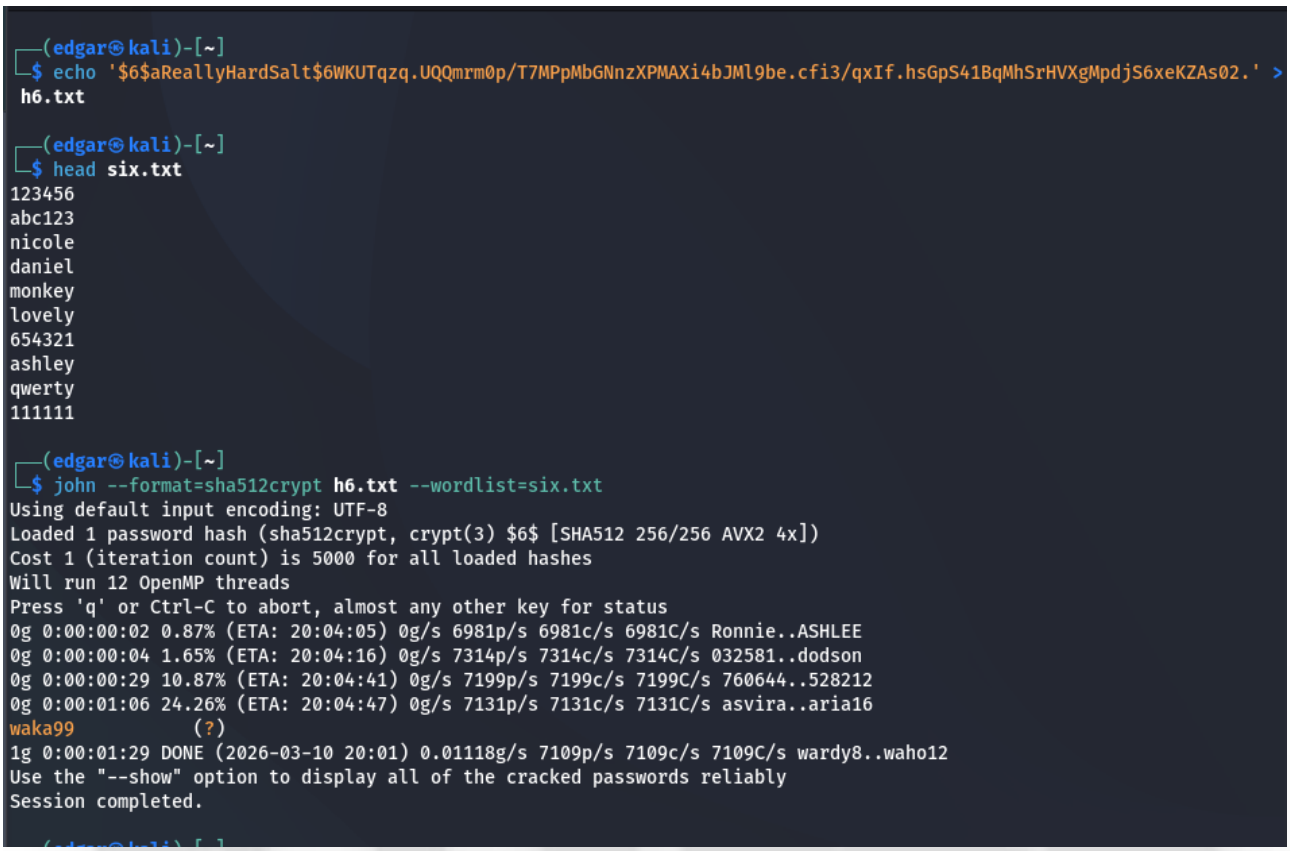
### Salt

`aReallyHardSalt`

### Method

First, I identified the hash format. The prefix `$6$` indicates a **SHA-512 crypt hash with salt**.

Next, I used **John the Ripper** with the **SHA-512 format** and the **rockyou wordlist**.




## Command Purpose

- **john** → used to perform password cracking.
- **--format=sha512crypt** → specifies the SHA-512 crypt hash format.
- **h6** → file containing the target hash.
- **--wordlist=rockyou.txt** → uses the rockyou password list to perform a dictionary attack.

## Answer

**waka99**

## Explanation

John the Ripper successfully cracked the hash using a dictionary attack with the rockyou wordlist. The recovered password was **waka99**, which exists in the wordlist despite the use of salting.

# Question 4

## Hash

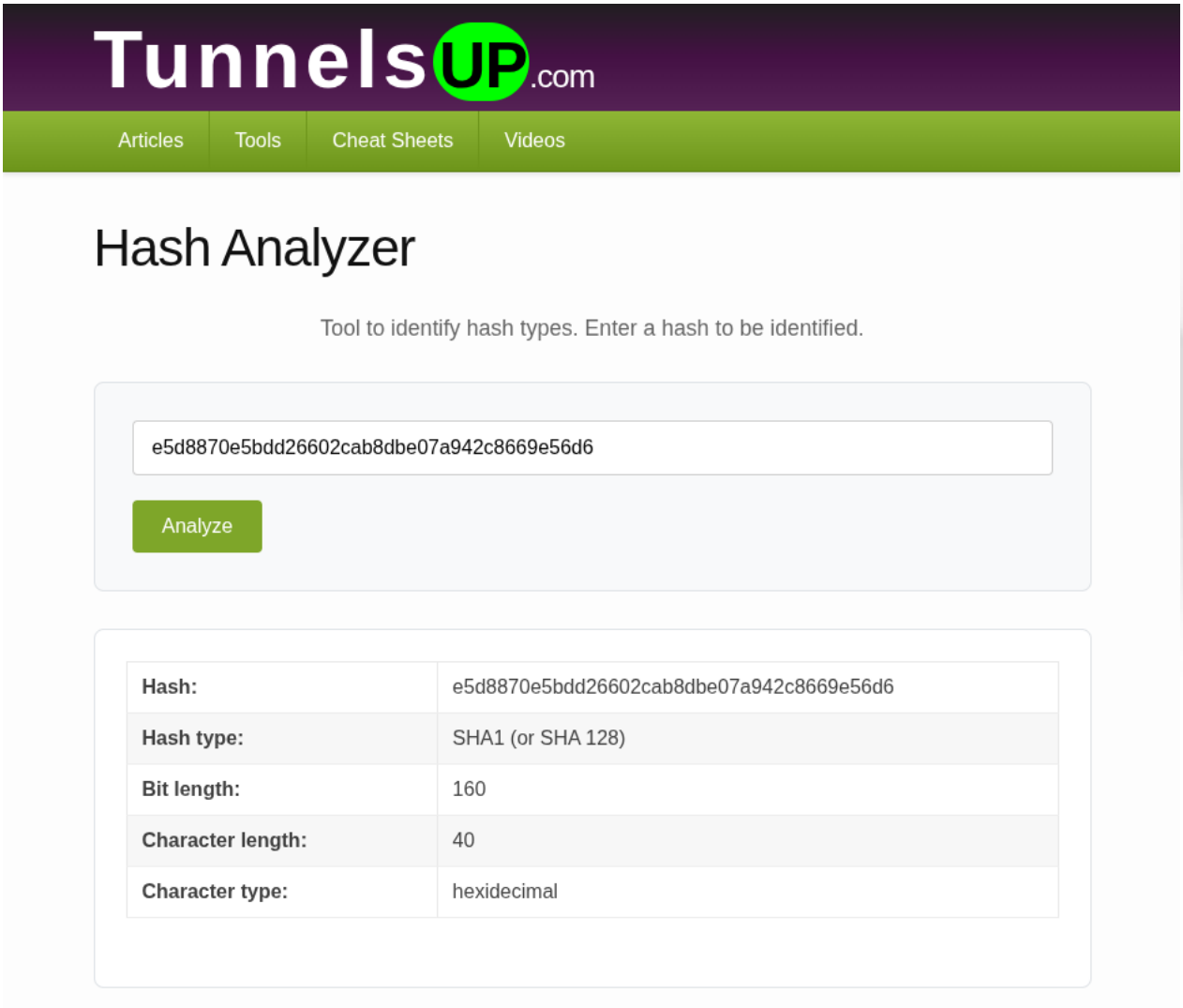
```
e5d8870e5bdd26602cab8dbe07a942c8669e56d6
```

## Salt

```
tryhackme
```

## Method

First, I used **TunnelsUp Hash Analyzer** to identify the hashing algorithm.



Then I prepared the hash and salt in the format required for **Hashcat**.

```
┌──(edgar㉿kali)-[~]
└─$ echo 'e5d8870e5bdd26602cab8dbe07a942c8669e56d6:tryhackme' > h7.txt

┌──(edgar㉿kali)-[~]
└─$ hashcat -a 0 -m 160 h7.txt /usr/share/wordlists/rockyou.txt --force
hashcat (v7.1.2) starting

You have enabled --force to bypass dangerous warnings and errors!
This can hide serious problems and should only be done when debugging.
Do not report hashcat issues encountered when using --force.

OpenCL API (OpenCL 3.0 PoCL 6.0+debian  Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEF, DISTRO, POCL_DEBUG) -
Platform #1 [The pocl project]
================================================================================================================================
* Device #01: cpu-haswell-Intel(R) Core(TM) i7-9750H CPU @ 2.60GHz, 6887/13775 MB (2048 MB allocatable), 12MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256
Minimum salt length supported by kernel: 0
Maximum salt length supported by kernel: 256

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Not-Iterated
* Single-Hash
* Single-Salt

ATTENTION! Pure (unoptimized) backend kernels selected.
Pure kernels can crack longer passwords, but drastically reduce performance.
If you want to switch to optimized kernels, append -O to your commandline.
See the above message to find out about the exact limits.

Watchdog: Temperature abort trigger set to 90c

Host memory allocated for this attack: 515 MB (12033 MB free)

Dictionary cache hit:
* Filename..: /usr/share/wordlists/rockyou.txt
* Passwords.: 14344385
* Bytes.....: 139921507
* Keyspace..: 14344385

e5d8870e5bdd26602cab8dbe07a942c8669e56d6:tryhackme:481616481616

Session..........: hashcat
Status...........: Cracked
Hash.Mode........: 160 (HMAC-SHA1 (key = $salt))
Hash.Target......: e5d8870e5bdd26602cab8dbe07a942c8669e56d6:tryhackme
Time.Started.....: Tue Mar 10 20:24:51 2026, (3 secs)
Time.Estimated...: Tue Mar 10 20:24:54 2026, (0 secs)
```

```
Kernel.Feature...: Pure Kernel (password length 0-256 bytes)
Guess.Base.......: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue......: 1/1 (100.00%)
Speed.#01........:  4474.1 kH/s (1.12ms) @ Accel:1024 Loops:1 Thr:1 Vec:8
Recovered........: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.........: 12324864/14344385 (85.92%)
Rejected.........: 0/12324864 (0.00%)
Restore.Point....: 12312576/14344385 (85.84%)
Restore.Sub.#01..: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#01...: 48162450 -> 4742264028
Hardware.Mon.#01.: Temp: 99c Util: 51%

Started: Tue Mar 10 20:24:38 2026
Stopped: Tue Mar 10 20:24:55 2026

┌──(edgar㉿kali)-[~]
└─$ hashcat --show -m 160 h7.txt
e5d8870e5bdd26602cab8dbe07a942c8669e56d6:tryhackme:481616481616
```

### Command Purpose

- **echo** → used to save the hash and salt in a file (`h7.txt`) in the correct format `hash:salt`.
- **hashcat -a 0** → runs a **dictionary attack**.
- **-m 160** → specifies the **SHA-1 salted hash mode**.
- **h7.txt** → file containing the target hash.
- **rockyou.txt** → password wordlist used for cracking.
- **--force** → forces Hashcat to run even if warnings appear.
- **hashcat --show** → displays the cracked password from the result.

### Answer

`481616481616`

### Explanation

Hashcat successfully matched the salted SHA-1 hash with the password **481616481616** using the **rockyou wordlist**. Even when salting is used, weak passwords can still be recovered through dictionary attacks if they exist in common password lists.

---

# Conclusion

This room from **Crack the Hash** provided practical experience with **hash identification and password cracking techniques**. Through the tasks, I learned how different hashing algorithms appear in practice and how to approach cracking them using both **online lookup services** and **dedicated cracking tools**.

During **Level 1**, the focus was on recognizing common hash formats and recovering weak passwords using public databases such as **CrackStation**. This demonstrated how frequently used passwords can often be recovered instantly if they already exist in large breach datasets.

In **Level 2**, the difficulty increased by requiring the use of professional tools such as **Hashcat** and **John the Ripper**. These tools allow security professionals to perform dictionary attacks using large password lists like **rockyou.txt**, which is commonly used in penetration testing environments.

One of the key lessons from this room is that **the security of stored credentials depends on both strong hashing algorithms and strong user passwords**. Even when hashing and salting are implemented, weak or commonly used passwords can still be recovered through dictionary attacks.

Overall, this room strengthened my understanding of:

- Identifying different hash types
- Using hash-cracking tools in Linux environments
- Performing dictionary attacks with wordlists
- Understanding the risks associated with weak passwords

These skills are essential for penetration testers when performing **credential analysis, password auditing, and post-exploitation activities** during security assessments.

---

**Final Notes**

This documentation is intentionally high-level and ethical.
It demonstrates learning outcomes and professional reasoning without exposing lab content, answers, or sensitive details.

---